

Nginx 静态资源服务与优化

一、学习目标

- 理解静态资源与动态资源的区别
 - 掌握 Nginx 静态资源服务配置方法
 - 学会配置 Gzip 压缩提升传输效率
 - 掌握浏览器缓存配置技巧
 - 了解防盗链配置保护资源
-

二、理论知识

2.1 静态资源与动态资源

在 Web 服务中，资源分为两大类：静态资源和动态资源。理解它们的区别是优化网站性能的基础。

静态资源

静态资源是指内容固定不变的文件，服务器直接读取文件内容返回给客户端，无需任何处理。常见的静态资源包括：HTML 文件、CSS 样式表、JavaScript 脚本、图片（JPG、PNG、GIF、SVG）、字体文件、视频和音频文件、PDF 文档等。

静态资源的特点是内容不会因请求者或请求时间而改变。就像书店里的书，每本书的内容都是固定的，谁来买看到的都是一样的内容。

动态资源

动态资源是指内容需要程序实时生成的资源，每次请求可能返回不同的内容。比如用户登录后看到的个人信息页面、购物车内容、搜索结果等。

动态资源需要后端程序（PHP、Python、Java 等）处理，根据用户身份、请求参数、数据库数据等生成响应内容。就像餐厅现做的菜，每位顾客点的菜可能不同，厨师需要根据订单现场制作。

Nginx 的角色

Nginx 在处理这两类资源时角色不同：

- 对于静态资源，Nginx 直接读取文件返回，效率极高；
 - 对于动态资源，Nginx 通常作为反向代理，将请求转发给后端应用服务器处理。
-

2.2 Nginx 处理静态资源的优势

Nginx 处理静态资源有以下显著优势：

高效的事件驱动架构

Nginx 使用异步非阻塞的事件驱动架构，单个 Worker 进程可以同时处理数千个并发连接。相比传统的多线程模型，资源消耗更低，性能更高。

sendfile 零拷贝技术

传统方式下，文件从磁盘到网络需要经过多次内存拷贝：

磁盘到内核缓冲区，

内核缓冲区到用户空间，

用户空间再到内核网络缓冲区。

Nginx 的 sendfile 功能跳过用户空间，直接在内核中完成文件到网络的传输，大幅减少 CPU 开销和内存拷贝。

高效的文件缓存

Nginx 可以将频繁访问的文件句柄和元数据缓存在内存中，减少磁盘 I/O 操作，提高响应速度。

2.3 Gzip 压缩原理

Gzip 是一种广泛使用的数据压缩格式，可以显著减小传输文件的大小，加快网页加载速度。

压缩流程：

当浏览器发送请求时，会在请求头中声明支持的压缩格式。Nginx 收到请求后，如果启用了 Gzip 并且文件类型匹配，就会将响应内容压缩后发送。浏览器收到压缩内容后自动解压显示。

请求头示例：

```
Accept-Encoding: gzip, deflate, br
```

响应头示例：

```
Content-Encoding: gzip
```

压缩效果：

Note

文本类资源（HTML、CSS、JavaScript、JSON、XML）压缩效果最好，通常可以减少 60%–80% 的体积。

图片、视频等已经压缩过的二进制文件压缩效果很差，不建议压缩。

2.4 浏览器缓存机制

合理配置浏览器缓存可以显著提升网站性能和用户体验。当用户再次访问网站时，浏览器可以直接使用本地缓存的资源，无需重新下载。

缓存控制头：

Cache-Control 是最常用的缓存控制头，常见取值包括：`public` 表示可以被任何缓存存储，`private` 表示只能被浏览器缓存，`no-cache` 表示使用前需验证，`no-store` 表示禁止缓存，`max-age=秒数` 表示缓存有效期。

Expires 指定资源的过期时间，是一个具体的日期时间。Cache-Control 的 max-age 优先级更高。

ETag 和 **Last-Modified** 用于缓存验证，当缓存过期后浏览器会携带这些信息询问服务器资源是否有更新，如果没有更新服务器返回 304 状态码，浏览器继续使用缓存。

缓存策略建议：

对于不经常变化的静态资源（如库文件、图片），可以设置较长的缓存时间（如一年），通过文件名加版本号或哈希值来更新缓存。对于经常更新的资源（如 HTML 页面），应设置较短的缓存时间或禁用缓存。

2.5 防盗链原理

防盗链是保护网站资源不被其他网站非法引用的技术。如果其他网站直接链接你的图片或视频，会消耗你的服务器带宽，却没有给你带来任何流量。

Referer 头

浏览器在请求资源时，会在 HTTP 头中包含 Referer 字段，标明请求来源页面的 URL。服务器可以检查这个字段来判断请求是否来自合法来源。

防盗链逻辑：

当请求到达 Nginx 时，检查 Referer 头。如果 Referer 为空（直接访问）或来自允许的域名，则正常返回资源。如果来自其他域名，则拒绝请求或返回替代内容（如警告图片）。

生活化类比：防盗链就像餐厅的会员验证，只有本店会员或合作商户的会员才能享受优惠，其他人不能使用。Referer 就像会员卡，用来证明你的来源身份。

注意事项：

Referer 头可以被伪造，所以防盗链不是绝对安全的，但可以防止大部分简单的盗链行为。有些浏览器或安全软件会阻止发送 Referer，需要考虑是否允许空 Referer。

三、实践内容

3.1 实验环境准备

实验目的：掌握静态资源服务配置和优化技巧

前置条件：已完成第2章实验，Nginx 已安装并配置了基本站点

3.2 实验一：创建完整的静态资源站点

操作步骤：

```
# 步骤1：创建目录结构
sudo mkdir -p /var/www/static-site/{css,js,images,fonts,downloads}

# 步骤2：创建示例 CSS 文件
sudo tee /var/www/static-site/css/style.css << 'EOF'
/* 主样式表 */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI',
    Roboto,
        'Helvetica Neue', Arial, sans-serif;
    line-height: 1.6;
    color: #333;
    background-color: #f5f5f5;
}

.container {
    max-width: 1200px;
    margin: 0 auto;
    padding: 20px;
}
```

```
header {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 40px 20px;
  text-align: center;
}

header h1 {
  font-size: 2.5rem;
  margin-bottom: 10px;
}

.content {
  background: white;
  padding: 30px;
  margin: 20px 0;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.feature-list {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  margin-top: 20px;
}

.feature-item {
  background: #f8f9fa;
  padding: 20px;
  border-radius: 8px;
  text-align: center;
}

.feature-item h3 {
  color: #667eea;
  margin-bottom: 10px;
}

footer {
  text-align: center;
  padding: 20px;
}
```

```
    color: #666;
}
EOF

# 步骤3: 创建示例 JavaScript 文件
sudo tee /var/www/static-site/js/main.js << 'EOF'
// 主脚本文件
document.addEventListener('DOMContentLoaded', function() {
    console.log('页面加载完成');

    // 显示当前时间
    function updateTime() {
        const timeElement = document.getElementById('current-time');
        if (timeElement) {
            const now = new Date();
            timeElement.textContent = now.toLocaleString('zh-CN');
        }
    }

    updateTime();
    setInterval(updateTime, 1000);

    // 添加点击事件
    const features = document.querySelectorAll('.feature-item');
    features.forEach(function(item) {
        item.addEventListener('click', function() {
            this.style.transform = 'scale(1.05)';
            setTimeout(() => {
                this.style.transform = 'scale(1)';
            }, 200);
        });
    });
});
EOF
```

```
# 步骤4: 创建主页 HTML
sudo tee /var/www/static-site/index.html << 'EOF'
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
<title>静态资源优化演示</title>
<link rel="stylesheet" href="/css/style.css">
</head>
<body>
  <header>
    <h1>🚀 Nginx 静态资源服务</h1>
    <p>高性能静态资源分发演示站点</p>
  </header>

  <div class="container">
    <div class="content">
      <h2>欢迎访问</h2>
      <p>当前时间: <span id="current-time">加载中...</span></p>
      <p>本站点演示了 Nginx 静态资源服务的各种优化技术, 包括: </p>

      <div class="feature-list">
        <div class="feature-item">
          <h3>📦 Gzip 压缩</h3>
          <p>减小传输体积, 加快加载速度</p>
        </div>
        <div class="feature-item">
          <h3>💾 浏览器缓存</h3>
          <p>合理设置缓存, 减少重复请求</p>
        </div>
        <div class="feature-item">
          <h3>🔒 防盗链</h3>
          <p>保护资源, 防止非法引用</p>
        </div>
        <div class="feature-item">
          <h3>⚡ sendfile</h3>
          <p>零拷贝技术, 高效文件传输</p>
        </div>
      </div>
    </div>
  </div>

  <div class="content">
    <h2>测试资源</h2>
    <p>以下是本站点的静态资源: </p>
    <ul>
```



```
        <li>CSS 文件: <a
href="/css/style.css">/css/style.css</a></li>
        <li>JS 文件: <a href="/js/main.js">/js/main.js</a>
</li>

        <li>测试图片: <a
href="/images/test.svg">/images/test.svg</a></li>
    </ul>
</div>
</div>

<footer>
    <p>Nginx 静态资源服务演示 &copy; 2025</p>
</footer>

<script src="/js/main.js"></script>
</body>
</html>
EOF
```

步骤5: 创建 SVG 测试图片

```
sudo tee /var/www/static-site/images/test.svg << 'EOF'
<svg xmlns="http://www.w3.org/2000/svg" width="200" height="200"
viewBox="0 0 200 200">
    <rect width="200" height="200" fill="#667eea"/>
    <circle cx="100" cy="100" r="60" fill="#764ba2"/>
    <text x="100" y="108" text-anchor="middle" fill="white" font-
size="24" font-family="Arial">TEST</text>
</svg>
EOF
```

步骤6: 设置权限

```
sudo chown -R nginx:nginx /var/www/static-site
sudo chmod -R 755 /var/www/static-site
```

步骤7: 验证目录结构

```
find /var/www/static-site -type f
```

3.3 实验二：配置基础静态资源服务

操作步骤：

步骤1: 创建站点配置文件

```
sudo tee /etc/nginx/conf.d/static-site.conf << 'EOF'
server {
    listen 8082;
    server_name localhost;

    # 网站根目录
    root /var/www/static-site;
    index index.html;

    # 字符编码
    charset utf-8;

    # 访问日志
    access_log /var/log/nginx/static-site_access.log main;
    error_log /var/log/nginx/static-site_error.log;

    # 根路径
    location / {
        try_files $uri $uri/ =404;
    }

    # 静态资源: CSS
    location /css/ {
        expires 7d;
        add_header Cache-Control "public, immutable";
    }

    # 静态资源: JavaScript
    location /js/ {
        expires 7d;
        add_header Cache-Control "public, immutable";
    }

    # 静态资源: 图片
    location /images/ {
        expires 30d;
        add_header Cache-Control "public";
    }

    # 静态资源: 字体
```

```
location /fonts/ {
    expires 365d;
    add_header Cache-Control "public, immutable";

    # 允许跨域请求字体
    add_header Access-Control-Allow-Origin *;
}
}
EOF
```

步骤2: 测试配置

```
sudo nginx -t
```

步骤3: 重新加载配置

```
sudo systemctl reload nginx
```

步骤4: 测试访问

```
curl -I http://localhost:8082/
```

```
curl -I http://localhost:8082/css/style.css
```

```
curl -I http://localhost:8082/images/test.svg
```

检查响应头:

查看 CSS 文件响应头, 观察缓存相关头信息

```
curl -I http://localhost:8082/css/style.css
```

预期看到类似:

Expires: ... (7天后的日期)

Cache-Control: public, immutable

3.4 实验三: 配置 Gzip 压缩

操作步骤:

步骤1: 查看未压缩时的响应

```
curl -I http://localhost:8082/css/style.css
```

注意 Content-Length 的值

步骤2: 修改配置添加 Gzip

```
sudo tee /etc/nginx/conf.d/static-site.conf << 'EOF'
server {
    listen 8082;
    server_name localhost;

    root /var/www/static-site;
    index index.html;
    charset utf-8;

    access_log /var/log/nginx/static-site_access.log main;
    error_log /var/log/nginx/static-site_error.log;

    # ===== Gzip 压缩配置 =====
    # 启用 Gzip
    gzip on;

    # 最小压缩文件大小 (小于此值不压缩)
    gzip_min_length 1024;

    # 压缩级别 1-9 (推荐 4-6)
    gzip_comp_level 5;

    # 需要压缩的文件类型
    gzip_types
        text/plain
        text/css
        text/xml
        text/javascript
        application/json
        application/javascript
        application/xml
        application/xml+rss
        application/x-javascript
        image/svg+xml;

    # 在响应头中添加 Vary: Accept-Encoding
    gzip_vary on;

    # 对所有代理请求启用压缩
    gzip_proxied any;
```

```
# 禁用对 IE6 的压缩（兼容旧浏览器）
```

```
gzip_disable "MSIE [1-6]\.";
```

```
# 压缩缓冲区
```

```
gzip_buffers 16 8k;
```

```
# ===== Gzip 配置结束 =====
```

```
location / {  
    try_files $uri $uri/ =404;  
}
```

```
location /css/ {  
    expires 7d;  
    add_header Cache-Control "public, immutable";  
}
```

```
location /js/ {  
    expires 7d;  
    add_header Cache-Control "public, immutable";  
}
```

```
location /images/ {  
    expires 30d;  
    add_header Cache-Control "public";  
}
```

```
location /fonts/ {  
    expires 365d;  
    add_header Cache-Control "public, immutable";  
    add_header Access-Control-Allow-Origin *;  
}
```

```
}
```

```
EOF
```

```
# 步骤3：测试并重载
```

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

```
# 步骤4：测试 Gzip 压缩效果
```

```
# 发送支持 gzip 的请求
```

```
curl -H "Accept-Encoding: gzip" -I
```

```
http://localhost:8082/css/style.css
```

```
# 预期看到:  
# Content-Encoding: gzip  
# Vary: Accept-Encoding
```

验证压缩效果:

```
# 步骤5: 对比压缩前后的大小  
# 不带 Accept-Encoding (不压缩)  
curl -so /dev/null -w "未压缩大小: %{size_download} bytes\n"  
http://localhost:8082/css/style.css  
  
# 带 Accept-Encoding (压缩)  
curl -so /dev/null -w "压缩后大小: %{size_download} bytes\n" -H  
"Accept-Encoding: gzip" http://localhost:8082/css/style.css
```

3.5 实验四: 配置详细的浏览器缓存策略

操作步骤:

```
# 步骤1: 创建分类缓存配置  
sudo tee /etc/nginx/conf.d/static-site.conf << 'EOF'  
server {  
    listen 8082;  
    server_name localhost;  
  
    root /var/www/static-site;  
    index index.html;  
    charset utf-8;  
  
    access_log /var/log/nginx/static-site_access.log main;  
    error_log /var/log/nginx/static-site_error.log;  
  
    # Gzip 压缩  
    gzip on;  
    gzip_min_length 1024;  
    gzip_comp_level 5;  
    gzip_types text/plain text/css text/xml text/javascript
```

```

        application/json application/javascript
application/xml
        application/xml+rss image/svg+xml;
gzip_vary on;
gzip_proxied any;

# ===== 缓存策略配置 =====

# HTML 文件：不缓存或短缓存
location ~* \.html$ {
    expires -1;
    add_header Cache-Control "no-cache, must-revalidate";
}

# CSS 和 JS：中等缓存时间
location ~* \.(css|js)$ {
    expires 7d;
    add_header Cache-Control "public, immutable";

    # 启用 ETag
    etag on;
}

# 图片：较长缓存时间
location ~* \.(jpg|jpeg|png|gif|ico|svg|webp)$ {
    expires 30d;
    add_header Cache-Control "public";

    # 关闭访问日志减少 I/O
    access_log off;
}

# 字体：最长缓存时间
location ~* \.(woff|woff2|ttf|eot|otf)$ {
    expires 365d;
    add_header Cache-Control "public, immutable";
    add_header Access-Control-Allow-Origin *;
    access_log off;
}

# 视频和音频
location ~* \.(mp4|webm|ogg|mp3|wav)$ {

```

```

        expires 30d;
        add_header Cache-Control "public";
    }

# 可下载文件
location ~* \.(pdf|doc|docx|xls|xlsx|zip|rar)$ {
    expires 7d;
    add_header Cache-Control "public";
}

# ===== 缓存策略结束 =====

# 默认处理
location / {
    try_files $uri $uri/ =404;
}
}
EOF

# 步骤2: 测试并重载
sudo nginx -t
sudo systemctl reload nginx

# 步骤3: 测试不同类型文件的缓存头
echo "=== HTML 文件 ==="
curl -I http://localhost:8082/index.html 2>/dev/null | grep -E
"Cache-Control|Expires"

echo "=== CSS 文件 ==="
curl -I http://localhost:8082/css/style.css 2>/dev/null | grep -E
"Cache-Control|Expires|ETag"

echo "=== 图片文件 ==="
curl -I http://localhost:8082/images/test.svg 2>/dev/null | grep -E
"Cache-Control|Expires"

```

3.7 实验五：配置防盗链

操作步骤：

步骤1: 创建防盗链警告图片

```
sudo tee /var/www/static-site/images/no-hotlink.svg << 'EOF'
<svg xmlns="http://www.w3.org/2000/svg" width="400" height="100"
viewBox="0 0 400 100">
  <rect width="400" height="100" fill="#e74c3c"/>
  <text x="200" y="60" text-anchor="middle" fill="white" font-
size="20" font-family="Arial">禁止盗链 - Hotlink Not Allowed</text>
</svg>
EOF
```

步骤2: 修改配置添加防盗链

```
sudo tee /etc/nginx/conf.d/static-site.conf << 'EOF'
server {
    listen 8082;
    server_name localhost;

    root /var/www/static-site;
    index index.html;
    charset utf-8;

    access_log /var/log/nginx/static-site_access.log main;
    error_log /var/log/nginx/static-site_error.log;

    # Gzip 压缩
    gzip on;
    gzip_min_length 1024;
    gzip_comp_level 5;
    gzip_types text/plain text/css text/xml text/javascript
        application/json application/javascript
application/xml
        application/xml+rss image/svg+xml;
    gzip_vary on;

    # 默认处理
    location / {
        try_files $uri $uri/ =404;
    }

    # CSS 和 JS
    location ~* \.(css|js)$ {
        expires 7d;
        add_header Cache-Control "public, immutable";
    }
}
```

```

    etag on;
}

# ===== 图片防盗链配置 =====
location ~* \.(jpg|jpeg|png|gif|ico|svg|webp)$ {
    expires 30d;
    add_header Cache-Control "public";
    access_log off;

    # 防盗链验证
    valid_referers none blocked server_names
        *.example.com
        localhost
        127.0.0.1
        ~\.google\.
        ~\.baidu\.;

    # 如果 referer 无效, 返回 403 或显示警告图片
    if ($invalid_referer) {
        # 方法1: 直接返回 403
        # return 403;

        # 方法2: 重写到警告图片
        rewrite ^/images/.*$ /images/no-hotlink.svg last;
    }
}

# ===== 防盗链配置结束 =====

# 字体
location ~* \.(woff|woff2|ttf|eot|otf)$ {
    expires 365d;
    add_header Cache-Control "public, immutable";
    add_header Access-Control-Allow-Origin *;
    access_log off;
}
}
EOF

# 步骤3: 测试并重载
sudo nginx -t
sudo systemctl reload nginx

```

```
# 步骤4: 测试防盗链
# 无 Referer (直接访问, 应该允许)
curl -I http://localhost:8082/images/test.svg

# 本站 Referer (应该允许)
curl -I -H "Referer: http://localhost:8082/"
http://localhost:8082/images/test.svg

# 外站 Referer (应该被拦截)
curl -I -H "Referer: http://evil-site.com/"
http://localhost:8082/images/test.svg
# 应该看到重定向到警告图片或返回 403
```

valid_referers 参数说明:

```
valid_referers none blocked server_names *.example.com;

# none - 允许没有 Referer 的请求 (直接访问、书签等)
# blocked - 允许 Referer 被防火墙删除的请求
# server_names - 允许当前 server_name 的请求
# *.example.com - 允许 example.com 子域名的请求
# ~\.google\. - 正则匹配, 允许 Google 的请求
```

3.8 实验六: 配置目录浏览 (autoindex)

有时需要允许用户浏览目录列表, 如下载站点。

操作步骤:

```
# 步骤1: 创建下载目录并添加文件
sudo mkdir -p /var/www/static-site/downloads
sudo tee /var/www/static-site/downloads/readme.txt << 'EOF'
这是一个示例下载文件。
用于演示 Nginx 目录浏览功能。
EOF

sudo tee /var/www/static-site/downloads/sample.txt << 'EOF'
Sample file content for testing.
```

EOF

```
sudo chown -R nginx:nginx /var/www/static-site/downloads
```

步骤2: 修改配置添加目录浏览

```
sudo tee -a /etc/nginx/conf.d/static-site.conf << 'EOF'
```

追加到原有配置后面 (如果手动编辑则添加到 server 块内)

EOF

完整重写配置

```
sudo tee /etc/nginx/conf.d/static-site.conf << 'EOF'
```

```
server {  
    listen 8082;  
    server_name localhost;  
  
    root /var/www/static-site;  
    index index.html;  
    charset utf-8;  
  
    access_log /var/log/nginx/static-site_access.log main;  
    error_log /var/log/nginx/static-site_error.log;  
  
    # Gzip 压缩  
    gzip on;  
    gzip_min_length 1024;  
    gzip_comp_level 5;  
    gzip_types text/plain text/css text/xml text/javascript  
        application/json application/javascript  
application/xml  
        application/xml+rss image/svg+xml;  
    gzip_vary on;  
  
    # 默认处理  
    location / {  
        try_files $uri $uri/ =404;  
    }  
  
    # CSS 和 JS  
    location ~* \.(css|js)$ {  
        expires 7d;  
        add_header Cache-Control "public, immutable";  
    }  
}
```

```

}

# 图片
location ~* \.(jpg|jpeg|png|gif|ico|svg|webp)$ {
    expires 30d;
    access_log off;
}

# ===== 下载目录浏览配置 =====
location /downloads/ {
    # 启用目录浏览
    autoindex on;

    # 显示文件大小 (on 显示精确字节, off 显示 K/M/G)
    autoindex_exact_size off;

    # 显示本地时间 (默认 GMT)
    autoindex_localtime on;

    # 输出格式: html (默认)、xml、json、jsonp
    autoindex_format html;

    # 设置默认 MIME 类型
    default_type application/octet-stream;

    # 添加 Content-Disposition 头触发下载
    add_header Content-Disposition "attachment";
}
# ===== 目录浏览配置结束 =====
}
EOF

# 步骤3: 测试并重载
sudo nginx -t
sudo systemctl reload nginx

# 步骤4: 测试目录浏览
curl http://localhost:8082/downloads/

# 步骤5: 在浏览器中访问
# http://YOUR_IP:8082/downloads/

```

3.9 实验七：配置 sendfile 和 tcp_nopush 优化

操作步骤：

```
# 步骤1：查看当前主配置中的相关设置
grep -E "sendfile|tcp_nopush|tcp_nodelay" /etc/nginx/nginx.conf

# 步骤2：创建优化配置（在 http 块级别）
# 这些配置通常放在 nginx.conf 中

# 备份原配置
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak2

# 步骤3：修改 nginx.conf 确保优化选项开启
sudo tee /etc/nginx/nginx.conf << 'EOF'
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;

include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
    use epoll;
    multi_accept on;
}

http {
    # 基本设置
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # 日志格式
    log_format main '$remote_addr - $remote_user [$time_local]
"$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;
```

```

# ===== 文件传输优化 =====
# 启用高效文件传输（零拷贝）
sendfile on;

# 在 sendfile 模式下，将响应头和文件内容一起发送
tcp_nopush on;

# 禁用 Nagle 算法，减少延迟
tcp_nodelay on;

# 连接超时
keepalive_timeout 65;

# 读取请求体超时
client_body_timeout 12;

# 发送响应超时
send_timeout 10;

# 客户端请求体缓冲区
client_body_buffer_size 16k;

# 客户端最大请求体大小
client_max_body_size 10m;
# ===== 优化配置结束 =====

# 文件类型哈希表大小
types_hash_max_size 4096;

# 引入站点配置
include /etc/nginx/conf.d/*.conf;
}
EOF

# 步骤4：测试并重载
sudo nginx -t
sudo systemctl reload nginx

# 步骤5：验证配置生效
sudo nginx -T | grep -E "sendfile|tcp_nopush|tcp_nodelay"

```

优化参数说明：

<code>sendfile on;</code>	开启零拷贝，避免文件内容在用户空间和内核空间之间复制
<code>tcp_nopush on;</code>	在 <code>sendfile</code> 模式下，将响应头和数据块一起发送，减少网络包数量
<code>tcp_nodelay on;</code>	禁用 Nagle 算法，适合小数据包实时传输

3.10 实验八：性能(压力)测试对比

操作步骤：

```
# 步骤1: 安装 Apache Benchmark 测试工具
sudo dnf install httpd-tools -y

# 步骤2: 测试静态页面性能
# 参数说明: -n 请求总数, -c 并发数
ab -n 1000 -c 10 http://localhost:8082/index.html

# 记录关键指标:
# - Requests per second (每秒请求数)
# - Time per request (平均响应时间)
# - Transfer rate (传输速率)

# 步骤3: 测试 CSS 文件 (带 Gzip)
ab -n 1000 -c 10 -H "Accept-Encoding: gzip"
http://localhost:8082/css/style.css

# 步骤4: 测试不带 Gzip 的请求
ab -n 1000 -c 10 http://localhost:8082/css/style.css

# 步骤5: 对比测试结果
# Gzip 版本传输量应该明显小于非 Gzip 版本

# 步骤6: 测试较高并发
ab -n 5000 -c 100 http://localhost:8082/index.html

# 步骤7: 查看 Nginx 状态 (需要启用 stub_status, 后续章节会讲)
```

测试结果分析：


```
# 示例输出关键信息：
# Concurrency Level:      10          # 并发数
# Time taken for tests:    1.234 sec   # 测试耗时
# Complete requests:      1000        # 完成请求数
# Failed requests:        0           # 失败请求数
# Total transferred:      256000 bytes # 总传输量
# Requests per second:    810.37 [#/sec] # 每秒请求数（重要！）
# Time per request:       12.340 [ms]  # 平均响应时间
# Transfer rate:          202.59 [KB/s] # 传输速率
```

3.11 实验九：创建静态资源配置模板

创建一个可复用的静态资源配置模板。

操作步骤：

```
# 步骤1：创建配置模板目录
sudo mkdir -p /etc/nginx/templates

# 步骤2：创建通用静态资源配置片段
sudo tee /etc/nginx/templates/static-optimization.conf << 'EOF'
# =====
# Nginx 静态资源优化配置模板
# 包含: Gzip 压缩、缓存策略、安全头
# 使用方法: 在 server 块中 include 此文件
# =====

# ----- Gzip 压缩配置 -----
gzip on;
gzip_min_length 1024;
gzip_comp_level 5;
gzip_types
    text/plain
    text/css
    text/xml
    text/javascript
    application/json
    application/javascript
    application/x-javascript
```

```
application/xml
application/xml+rss
application/atom+xml
image/svg+xml
font/opentype
font/ttf
font/woff
font/woff2;

gzip_vary on;
gzip_proxied any;
gzip_disable "MSIE [1-6]\.";
gzip_buffers 16 8k;

# ----- 安全响应头 -----
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-XSS-Protection "1; mode=block" always;

# ----- HTML 文件（不缓存） -----
location ~* \.html$ {
    expires -1;
    add_header Cache-Control "no-cache, must-revalidate";
    add_header X-Content-Type-Options "nosniff" always;
}

# ----- CSS 文件 -----
location ~* \.css$ {
    expires 7d;
    add_header Cache-Control "public, immutable";
    access_log off;
}

# ----- JavaScript 文件 -----
location ~* \.js$ {
    expires 7d;
    add_header Cache-Control "public, immutable";
    access_log off;
}

# ----- 图片文件 -----
location ~* \.(jpg|jpeg|png|gif|ico|svg|webp|bmp|tiff)$ {
    expires 30d;
```

```

    add_header Cache-Control "public";
    access_log off;
}

# ----- 字体文件 -----
location ~* \.(woff|woff2|ttf|eot|otf)$ {
    expires 365d;
    add_header Cache-Control "public, immutable";
    add_header Access-Control-Allow-Origin "*";
    access_log off;
}

# ----- 视频音频文件 -----
location ~* \.(mp4|webm|ogg|mp3|wav|flac|aac)$ {
    expires 30d;
    add_header Cache-Control "public";
}

# ----- 文档文件 -----
location ~* \.(pdf|doc|docx|xls|xlsx|ppt|pptx|zip|rar|7z|gz|tar)$ {
    expires 7d;
    add_header Cache-Control "public";
}
EOF

```

步骤3: 创建使用模板的站点配置

```

sudo tee /etc/nginx/conf.d/static-site.conf << 'EOF'
server {
    listen 8082;
    server_name localhost;

    root /var/www/static-site;
    index index.html;
    charset utf-8;

    access_log /var/log/nginx/static-site_access.log main;
    error_log /var/log/nginx/static-site_error.log;

    # 引入静态资源优化配置
    include /etc/nginx/templates/static-optimization.conf;

    # 默认处理

```

```
location / {
    try_files $uri $uri/ =404;
}

# 下载目录
location /downloads/ {
    autoindex on;
    autoindex_exact_size off;
    autoindex_localtime on;
}
}
EOF

# 步骤4：测试并重载
sudo nginx -t
sudo systemctl reload nginx

# 步骤5：验证配置
curl -I http://localhost:8082/css/style.css
curl -I -H "Accept-Encoding: gzip" http://localhost:8082/js/main.js
```

四、本章小结

通过本章学习，我们掌握了以下知识点：

理论知识：

1. 静态资源与动态资源的区别及 Nginx 的处理方式
2. sendfile 零拷贝技术原理
3. Gzip 压缩的工作原理和适用场景
4. 浏览器缓存机制和 Cache-Control、Expires、ETag 等头信息
5. 防盗链的 Referer 验证原理

实践技能：

1. 创建结构化的静态资源网站
2. 配置 Gzip 压缩并验证效果

3. 针对不同资源类型配置缓存策略
4. 配置 ETag 和条件请求验证
5. 配置防盗链保护图片资源
6. 配置目录浏览功能
7. 优化 sendfile、tcp_nopush、tcp_nodelay
8. 使用 ab 工具进行性能测试
9. 创建可复用的配置模板